Αναγνώριση Προτύπων - Εργαστήριο 1

Αναγνώριση φωνής με Κρυφά Μαρκοβιανά Μοντέλα και Αναδρομικά Νευρωνικά Δίκτυα

Κόρδας Νικόλαος (03121032)
Κριθαρίδης Κωνσταντίνος (03121045)

November 13, 2025

Βήμα 9

Πρώτα από όλα κάνουμε clone τα δεδομένα σε μορφή .wav από το repository της εργασίας στον υπολογιστή μας.

Στη συνέχεια, αξιοποιούμε τον βοηθητικό κώδικα του εργαστηρίου προκειμένου να υλοποιήσουμε τα επόμενα βήματά μας (*parser.py, hmm.py*)

```
def create_data():
    X, X_test, y, y_test, spk, spk_test = parser("recordings", n_mfcc=13)

    # TODO: YOUR CODE HERE
    (
        X_train,
        X_val,
        y_train,
        y_val,
        spk_train,
        spk_val,
    ) =  train_test_split(X, y, spk, test_size=0.2, random_state=42, stratify=y
    ) # split X into a 80/20 train validation split

    # Scaling data based on the training set
    scale_fn = make_scale_fn(X_train)
    X_train = scale_fn(X_train)
    X_val = scale_fn(X_val)
    X_test = scale_fn(X_test)

    train_dic = gather_in_dic(X_train, y_train, spk_train)
    val_dic = gather_in_dic(X_val, y_val, spk_val)
    test_dic = gather_in_dic(X_test, y_test, spk_test)
    labels = list(set(y_train))

    return train_dic, y_train, val_dic, y_val, test_dic, y_test, labels
```

Όπως φαίνεται και στη συνάρτηση παραπάνω, διαβάζουμε από το φάκελο τα wav αρχεία και εξάγουμε 13 MFCC για το καθένα. Στη συνέχεια με την έτοιμη συνάρτηση split του κώδικα τη χωρίζουμε στο default train test split το οποίο έχει 2700 από τα 3000 δείγματα ακολουθιών στο train και τα υπόλοιπα στο test. Στη συνέχεια, καλούμαστε να χωρίσουμε το test set αυτό σε 2 μικρότερα. Ένα validation και ένα test set σε αναλογία 80-20 αντίστοιχα. Αυτό, όπως φαίνεται και στον κώδικα παραπάνω, γίνεται με τη χρήση του *train_test_split*. Η παράμετρος stratify μας εξασφαλίζει την ομοιόμορφη κατανομή των δειγμάτων. Δηλαδή, ότι θα έχουμε τον ίδιο αριθμό ψηφίων από κάθε κατηγορία. Έτσι αποφεύγουμε biases.



Figure 1: Ανάγνωση Dataset

Βήμα 10

Στο βήμα αυτό υλοποιούμε αναγνώριση ψηφίων από ομιλία με GMM-HMM. Η λειτουργία του μοντέλου φαίνεται διαγραμματικά παρακάτω:

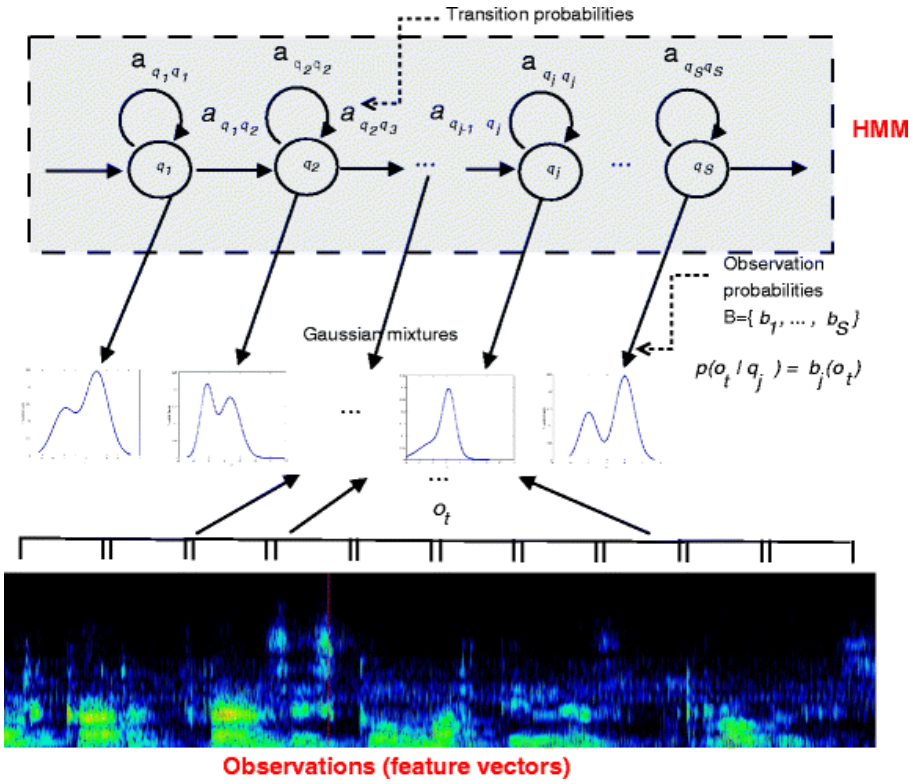


Figure 2: Image by [1]

Δημιουργούμε ένα HMM ανά ψηφίο, οπότε η έξοδος του i-οστού HMM θα είναι η πιθανότητα το σήμα να αντιστοιχεί στο i-οστό ψηφίο. Οι παρατηρήσεις σε κάθε state του HMM είναι ένα διαφορετικό mixture of gaussians. Αφού εκπαιδευτεί το GMM από μόνο του σε όλα τα δεδομένα, θα εκπαιδευτεί από κοινού το GMM-HMM μοντέλο και το μείγμα κατανομών ανά state θα γίνει fit στην παρατήρηση που συμβαίνει σε αυτό το state. Ιδανικά, θα θέλαμε εξ'αρχής να μην εκπαιδεύουμε το GMM σε όλα τα δεδομένα, αλλά μόνο σε αυτά που αντιστοιχούν στο χρονικό διάστημα που περιγράφεται από το state. Κάτι τέτοιο, όμως, είναι δυστυχώς δύσκολο να γίνει, αφού δεν είναι προκαθορισμένο για κάθε ψηφίο το πότε θα γίνει η μετάβαση από το ένα state στο επόμενο, οπότε θα αρκεστούμε στην διαφοροποίηση μεταξύ των GMMs που κάνει στη συνέχεια η από κοινού εκπαίδευση. Τέλος, σημειώνουμε ότι με το μείγμα γκαουσιανών μοντελοποιούμε την κατανομή των MFCCs ανά ψηφίο και state, αφού περιμένουμε πως η κάθε τέτοια κατανομή δεν θα είναι unimodal στον χώρο των MFCCs λόγω των διαφορών στην εκφώνηση του ίδιου ψηφίου μεταξύ διαφορετικών ομιλητών.

Η συνάρτηση train φαίνεται παρακάτω:

```
1 def train_hmms(train_dic, labels, n_states, n_mixtures):
2     hmms = {}  # create one hmm for each digit
3
```

```
    for dig in labels:
        X, _, _, _ = train_dic[dig]
        # TODO: YOUR CODE HERE
        emission_model = None
        if gmm: emission_model = initialize_and_fit_gmm_distributions(X,
    n_states=n_states, n_mixtures=n_mixtures)
        else: emission_model = initialize_and_fit_normal_distributions(X,
    n_states=n_states)
        hmms[dig] = train_single_hmm(X, emission_model, dig, n_states)
    return hmms
```

Την `initialize_and_fit_normal_distributions` καθώς είναι υποπερίπτωση του GMM για n_mixtures $= 1$.

Η αρχική εκπαίδευση του GMM γίνεται ως εξής:

```
def initialize_and_fit_gmm_distributions(X, n_states, n_mixtures):
    # TODO: YOUR CODE HERE
    dists = []
    all_data = np.concatenate(X, dtype=np.float32)
    D = all_data.shape[1]

    global_mean = np.mean(all_data, axis=0)
    global_std = np.std(all_data, axis=0)

    variance_floor = 1e-6
    base_variance = np.maximum(global_std**2, variance_floor).astype(np.float32
    )

    for _ in range(n_states):
        distributions = []
        for _ in range(n_mixtures):
            means = global_mean + 0.1 * np.random.randn(D).astype(np.float32)

            covs = base_variance.copy()

            distributions.append(
                Normal(means=means, covs=covs, covariance_type=covariance_type)
            )

        gmm_model = GeneralMixtureModel(
            distributions,
            verbose=False,
            max_iter=n_iters,
            random_state=42,
            tol=tol
        ).fit(all_data)
        dists.append(gmm_model)

    return dists
```

Δηλαδή αρχικοποιούμε κάθε γκαουσιανή σε multivariate gaussian με μέσο ειλημμένο από D-διάστατη τυπική κανονική κατανομή, όπου D η διάσταση των δεδομένων, εδώ 13 για τα MFCCs, και με συνδιασπορά τον D-διάστατο ταυτοτικό πίνακα. Επίσης περιορίζουμε κατά την εκπαίδευση τον πίνακα συνδιασποράς να είναι διαγώνιος, ώστε να μειώσουμε τις παραμέτρους και τον χρόνο εκπαίδευσης αλλά και να έχουμε πιο απλές αναπαραστάσεις για τα δεδομένα μας.

Αντίστοιχα, το HMM το εκπαιδεύουμε ως εξής:

```
def train_single_hmm(X, emission_model, digit, n_states):
    A = initialize_transition_matrix(n_states)
    start_probs = initialize_starting_probabilities(n_states)
    end_probs = initialize_end_probabilities(n_states)
```

```
    data = [x.astype(np.float32) for x in X]

    model = DenseHMM(
        distributions=emission_model,
        edges=A,
        starts=start_probs,
        ends=end_probs,
        verbose=True,
        max_iter=n_iters,
        tol=tol,
        random_state=42
    ).fit(data)
    return model
```

Για πίνακα μετάβασης χρησιμοποιήσαμε τον $A = [\alpha_{i,j}]_{1 \leq i,j \leq n}$, όπου $n = \text{n_states}$ το πλήθος καταστάσεων στα HMMs, με:

$$\alpha_{i,j} = \begin{cases} 1, & \text{if } j = i = n \\ 0.8 & \text{if } j = i \neq n \\ 0.2 & \text{if } j = i + 1 \\ 0 & \text{αλλιώς} \end{cases}$$

Οι τιμές 0.8 για την ανακύκλωση και 0.2 για την μετάβαση επιλέχθηκαν επειδή το μοντέλο πιο συχνά θα επιλέγει να μείνει στην ίδια κατάσταση, παρά να μεταβεί στην επόμενη.

```
def initialize_transition_matrix(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    A = np.full((n_states, n_states), 1e-6 ,dtype=np.float32)
    for i in range(n_states):
        if i+1 < n_states:
            A[i][i] = np.float32(0.8)
            A[i][i+1] = np.float32(0.2)
        else:
            A[i][i] = np.float32(1.0)
    return A
```

Τέλος, αρχικοποιήσαμε τις πιθανότητες εκκίνησης ως 1 στην αρχική κατάσταση και 0 σε όλες τις άλλες, ενώ τις πιθανότητες τερματισμού ως 1 στην τελική κατάσταση και 0 σε όλες τις άλλες.

```
def initialize_starting_probabilities(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    pi = np.zeros(n_states, dtype=np.float32)
    pi[0] = np.float32(1.0)
    return pi


def initialize_end_probabilities(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    pi_end = np.zeros(n_states, dtype=np.float32)
    pi_end[-1] = np.float32(1.0)
    return pi_end
```

Βήμα 11

Πλέον, έχουμε υλοποιήσει τα μοντέλα, οπότε μπορούμε να τα εκπαιδεύσουμε στο training set και να ελέγξουμε την απόδοση του καθενός στο validation set. Ελέγχουμε την απόδοση του μοντέλου για πλήθος καταστάσεων από 1 έως 4 και πλήθος γκαουσιανών στο μείγμα από 1 έως

Βήμα 12

Βήμα 13

Βήμα 14

1

Η υλοποίηση του βήματος αυτού βασίζεται στη δομή του βοηθητικού κώδικα για το εργαστήριο και συγκεκριμένα στο αρχείο *lstm.py*. Έχουν συμπληρωθεί τα τμήματα σημασμένα ως TODO και επιπλέον έχουν τροποποιηθεί λειτουργικότητες και προστεθεί επιπλεόν συναρτήσεις ώστε να καλυφθούν τα υποερωτήματα του βήματος.

2

Για την κατασκευή του LSTM δικτύου αξιοποιήσαμε τη βιβλιοθήκη PyTorch. Ο κώδικας φαίνεται παρακάτω:

```
class BasicLSTM(nn.Module):
    def __init__(
        self,
        input_dim,
        rnn_size,
        output_dim,
        num_layers,
        bidirectional=False,
        dropout=dropout,
    ):
        super(BasicLSTM, self).__init__()
        self.bidirectional = bidirectional
        self.feature_size = rnn_size * 2 if self.bidirectional else rnn_size

        # TODO: YOUR CODE HERE
        # --------------- Insert your code here ---------------- #
        # Initialize the LSTM, Dropout, Output layers
        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=rnn_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0,
            bidirectional=bidirectional,
        )

        self.dropout = nn.Dropout(dropout)
        self.output_layer = nn.Linear(self.feature_size, output_dim)


    def forward(self, x, lengths):
        """
        x : 3D numpy array of dimension N x L x D
            N: batch index
            L: sequence index
            D: feature index

        lengths: N x 1
        """

        # TODO: YOUR CODE HERE
        # --------------- Insert your code here ---------------- #
        if not isinstance(x, torch.Tensor):
```

```
            x = torch.tensor(x, dtype=torch.float32)
        if not isinstance(lengths, torch.Tensor):
            lengths = torch.tensor(lengths, dtype=torch.int64)

        if use_packing:
            # sort the features in decreasing length for packing
            lengths_sorted, sort_idx = torch.sort(lengths, descending=True)
            x_sorted = x[sort_idx]
      # pack sequence to avoid processing the padding
            packed_input = pack_padded_sequence(x_sorted, lengths_sorted.cpu(),
    batch_first=True, enforce_sorted=True)
            # get packed output
            packed_output, _ = self.lstm(packed_input)
      # retrieve sequence in its original (padded) format
            outputs, _ = pad_packed_sequence(packed_output, batch_first=True)
            # retrieve the output in its original order
            _, unsort_idx = torch.sort(sort_idx)
            outputs = outputs[unsort_idx]
        else:
            outputs, _ = self.lstm(x)

        # You must have all of the outputs of the LSTM, but you need only the
    last one (that does not exceed the sequence length)
        # To get it use the last_timestep method
        # Then pass it through the remaining network
        last_outputs = self.last_timestep(outputs, lengths, self.bidirectional)

        last_outputs = self.dropout(last_outputs)
        last_outputs = self.output_layer(last_outputs)

        return last_outputs
```

Κατά την αρχικοποίηση του δικτύου ελέγχεται κατά πόσο η αρχιτεκτονική του είναι αμφίδρομη ή όχι. Εφόσον είναι, αφενός τίθεται σε True η αντίστοιχη παράμετρος και αφετέρου φροντίζουμε ώστε να διπλασιαστεί το μέγεθος του hidden state του δικτύου.

Στη συνέχεια, επισημαίνουμε τα είδη των layers που θα χρησιμοποιηθούν στην υλοποίηση του δικτύου. Το κυρίως layer είναι το nn.LSTM, το οποίο ρυθμίζεται με τις παραμέτρους του με τρόπο τέτοιο ώστε να καλύψει τις απαιτήσεις της εκφώνησης στα πιθανά ερωτήματα που θα προκύψουν. Το dropout layer δεν χρησιμοποιείται πάντα. Αυτό το καθορίζουμε με την παράμετρο dropout του nn.Dropout layer. Εάν αυτή είναι 0 προφανώς αυτή η τεχνική regularization δεν χρησιμοποιείται. Επιπλέον, αξίζει να αναφέρουμε ότι έχουμε φροντίσει το Dropout να μην εφαρμοστεί στο τελευταίο layer. Το nn.Linear layer χρησιμοποιείται ώστε να πάρει τα output του LSTM (για αυτό έχει και μέγεθος όσο του hidden state) και να τα συνδυάσει προκειμένου να πάρει την τελική απόφαση για το classification του ψηφίου.

Η συνάρτηση forward είναι αυτή που καθορίζει το πώς γίνεται το inference του μοντέλου. Δηλαδή, κατ' επέκταση, τη διάταξη των δομικών layer που ορίσαμε και εξηγήσαμε παραπάνω. Ο πρώτος διαχωρισμός που κάνουμε έγκειται στο αν χρησιμοποιούμε την τεχνική packing στο input (χρειάζεται για το ερώτημα 8 - Bonus). Με την τεχνική του padding πετυχαίνουμε 2 πράγματα. Αρχικά, σε dataset με δεδομένα ακολουθίες διαφορετικών μηκών, είναι σκόπινο να ενώσουμε τις μικρότερες από αυτές ώστε να τις επεξεργαζόμαστε ταυτόχρονα και να εξοικονομούμε χρόνο κατά την εκπαίδευση. Επιπρόσθετα, αποφεύγουμε το zero-padding και την επεξεργασία αυτής της παραμορφωμένης εκδοχής των ακολουθιών εισόδου. Αυτή η τεχνική είναι ιδιαίτερα χρήσιμη σε δίκτυα στα οποία το μέγεθος εισόδου είναι αυστηρά καθορισμένο, όπως οι Transformers. Ο τρόπος που επιλέγουμε να εκτελέσουμε το packing είναι αρχικά να ταξινομήσουμε τις ακολουθίες με τα φωνήματα κατά φθίνον μήκος έτσι ώστε να τις συνδυάσουμε αποτελεσματικότερα. Αυτός ο συνδυασμός πραγματοποιοείται από την συνάρτηση *packed_padded_sequence()* της βιβλιοθήκης της Pytorch (βλέπουμε ότι το

enforced_sorted είναι ορισμένο σε True).

Τέλος, όπως βλέπει κανείς εδώ:

```
        last_outputs = self.last_timestep(outputs, lengths, self.bidirectional)

        last_outputs = self.dropout(last_outputs)
        last_outputs = self.output_layer(last_outputs)
```

```
    def last_timestep(self, outputs, lengths, bidirectional=False):
        """
        Returns the last output of the LSTM taking into account the zero
    padding
        """
        # TODO: READ THIS CODE AND UNDERSTAND WHAT IT DOES
        if bidirectional:
            forward, backward = self.split_directions(outputs)
            last_forward = self.last_by_index(forward, lengths)
            last_backward = backward[:, 0, :]
            # Concatenate and return - maybe add more functionalities like
    average
            return torch.cat((last_forward, last_backward), dim=-1)

        else:
            return self.last_by_index(outputs, lengths)
```

Έτσι εξασφαλίζουμε ότι το dropout δε θα μπει στο τελευταίο layer.

3

Σε αυτό το βήμα καλούμαστε να εκπαιδεύσουμε το μοντέλο. Έτσι, αρχικά, θα εξηγήσουμε τη διαδικασία της εκπαίδευσής του.

```
def training_loop(model, train_dataloader, optimizer, criterion):
    model.train()
    running_loss = 0.0
    num_batches = 0
    for num_batch, batch in enumerate(train_dataloader):
        features, labels, lengths = batch
        # TODO: YOUR CODE HERE
        features = features.to(device).float()
        labels = labels.to(device)
        lengths = lengths.to(device)

        # zero grads in the optimizer
        optimizer.zero_grad()
        # run forward pass
        outputs = model(features, lengths)
        # calculate loss
        loss = criterion(outputs, labels)

        # TODO: YOUR CODE HERE
        # Run backward pass
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        num_batches += 1
    train_loss = running_loss / num_batches
    return train_loss
```

Στο trainning loop παίρνουμε τα δεδομένα του dataloader batch by batch, υπολογίζουμε το loss και κάνουμε το ανάλογο βήμα στον optimizer.

Ο optimizer που χρησιμοποιούμε είναι ο AdamW. Αξιοποιεί όρους ορμής και έχει μοντέλο για να μεταβάλει τους όρους του learning rate και της ταχύτητας κατάλληλα ώστε να επιτύχει την καλύτερη σύγλιση. Προφανώς εκτυπώνουμε το loss σε κάθε εποχή όπως μας ζητείται στην εκφώνηση.

4

Στην συνάρτηση train:

```
    metrics = {
        "training_loss": {},
        "validation_loss": {},
        "validation_accuracy": {},
    }

    confusion_matrices = {}

    best_model_name, best_val_acc = None, -1

    for model_name, model, optimizer in zip(models.keys(), models.values(),
    optimizers.values()):
        model.to(device)

        early_stopping = EarlyStopping(patience, mode="min", base_model=None)

        print(f"Training in {device}")

        metrics["training_loss"][model_name] = []
        metrics["validation_loss"][model_name] = []
        metrics["validation_accuracy"][model_name] = []
        for epoch in range(epochs):
            training_loss = training_loop(model, train_dataloader, optimizer,
    criterion)
            valid_loss, y_pred, y_true = evaluation_loop(model, val_dataloader,
     criterion)

            # TODO: Calculate and print accuracy score
            valid_accuracy = accuracy_score(y_true, y_pred)
            print(
                "Epoch {}: train loss = {}, valid loss = {}, valid acc = {}".
    format(
                    epoch, training_loss, valid_loss, valid_accuracy
                )
            )
            metrics["training_loss"][model_name].append(training_loss)
            metrics["validation_loss"][model_name].append(valid_loss)
            metrics["validation_accuracy"][model_name].append(valid_accuracy)

            if valid_accuracy > best_val_acc:
                best_model_name = model_name

            if early_stopping.stop(valid_loss, model):
                print("early stopping...")
                model_weights = torch.load(checkpoint_filename, weights_only=
    False)
                break

    return best_model_name, metrics, models

def master_train(train_dataloader, val_dataloader, criterion):
```

```
47      global use_early_stopping
```

Παίρνουμε τις μετρικές που μας ενδιαφέρουν, δηλαδή το trainning και validation loss καθώς και το validation accuracy. Η master train είναι αυτή που εκπαιδεύει όλα τα μοντέλα που μας ενδιαφέρουν.

5

Καλούμαστε να προσθέσουμε L2 Regularization στο μοντέλο μας, καθώς και dropout.

Το L2 regularization δεν είναι κάτι το οποίο επηρεάζει δομικά το μοντέλο μας άμεσα. Είναι ένας όρος ποινικοποίησης του μέτρου των βαρών που προστίθεται στο loss function. Έτσι, μειώνουμε την πολυπλοκότητα του μοντέλου και έχουμε ως αποτέλεσμα την αποφυγή του overfitting. Αυτό είναι σημαντικό και αναμένεται να παίξει ρόλο, ειδικά σε ένα απλό dataset όπως είναι το δικό μας. Ένα επιπλέον πράγμα που πετυχαίνουμε είναι να αποφύγουμε το saturation στα βάρη του δικτύου μας και να ενισχύσουμε έτσι τη δυνατότητα generalization του μοντέλου μας.

Η τεχνική dropout απενεργοποιεί ένα ποσοστό νευρώνων (καθορίζεται από την παράμετρο dropout) κατά τη διάρκεια της εκπαίδευσης έτσι ώστε να σιγουρευτούμε ότι όλοι θα εκπαιδευτούν και έτσι θα αποφύγουμε overfitting και το coadaptation. Προφανώς η απενεργοποίηση αυτή είναι προσωρινή και ο νευρώνας δυνητικά επανέρχεται στην επόμενη εποχή (απενεργοποιούνται άλλοι).

Τα χαρακτηριστικά που επιλέγουμε στην εκπαίδευση του μοντέλου μας είναι:

- epochs: 750
- learning rate: $2 * 10^{-5}$
- dropout: 0.4

Ο λόγος που επιλέγουμε μικρό learning rate είναι πως κατά τη διάρκεια της εκπαίδευσης παρατηρούσαμε ταλάντωση της μετρικής accuracy για μεγαλύτερα. Έτσι, οδηγηθήκαμε και στο σχετικά μεγάλο εποχών (750), αφού λόγω του μικρού ρυθμού μάθησης το μοντέλο αργεί να συγκλίνει. Σταματήσαμε σε αυτόν το αριθμό διότι από εκεί και πάνω δεν παρατηρούσαμε καμία βελτίωση.

6

Αρχικά, θα εξηγήσουμε τη σημασία του early stopping και κατόπιν θα προχωρήσουμε στην ανάλυση της δικής μας υλοποίησης.

Με το early stopping διακόπτουμε πρόωρα την εκπαίδευση του μοντέλου (πριν τη συμπλήρωση του προαπαιτούμενου αριθμού εποχών). Αυτό γίνεται με αυτό ματο τρόπο. Ουσιαστικά, αν συμπληρωθεί ένας ορισμένος αριθμός εποχών (εδώ patience είναι 10) στον οποίο το μοντέλο μας δεν βελτιώνεται, σταματάμε. Με αυτό τον τρόπο πετυχαίνουμε τα εξής:

- Αποτρέπει το overfitting
- Εξοικονομεί χρόνο και πόρους. Αν το μοντέλο έχει πρακτικά συγκλίνει σε μικρότερο αριθμό εποχών από αυτόν που αναμένουμε, δεν συνεχίζουμε άσκοπα.

Στον κώδικά μας χρησιμοποιούμε την κλάση EarlyStopping:

```
1
2 class EarlyStopping(object):
3     def __init__(self, patience, mode="min", base=None, base_model=None):
4         self.best = base
5         self.patience = patience
```

```
        self.patience_left = patience
        self.mode = mode
        self.pickle = None if base_model == None else torch.save(base_model.
    state_dict(), checkpoint_filename)

    def stop(self, value: float, model) -> bool:
        # TODO: YOUR CODE HERE
        # Decrease patience if the metric hs not improved
        if self.has_improved(value):
            self.best = value
            self.patience_left = self.patience  # reset patience
            self.model = model
            self.pickle = torch.save(model.state_dict(), checkpoint_filename)
        else:
            self.patience_left -= int(use_early_stopping)

        # Stop when patience reaches zero
        return self.patience_left <= 0

    def has_improved(self, value: float) -> bool:
        # TODO: YOUR CODE HERE
        # Check if the metric has improved
        if self.best is None:
            return True

        if self.mode == "min":
            return value < self.best
        elif self.mode == "max":
            return value > self.best
        else:
            raise ValueError(f"Invalid mode '{self.mode}'. Use 'min' or 'max'."
    )
```

Κεντρική μέθοδός της είναι η stop. Ουσιαστικά ελέγχει κατά πόσο έχει βελτιωθεί το μοντέλο μας σε σχέση με την προηγούμενη εποχή (το συγκρίνει με το μέχρι τώρα καλύτερο) και εφόσον δεν το έχει κάνει αφαιρεί μία εποχή από το περιθώριο υπομονής μας (*patience_left*). Όταν εξαντληθεί αυτό, δηλαδή φτάσουμε στο 0, αποθηκεύουμε το μοντέλο μας σε pickle format με τις κατάλληλες συναρτήσεις της PyTorch.

7

Σε αυτό το υποερώτημα καλούμαστε να πειραματιστούμε και με ένα bidirectional LSTM. Όπως έχουμε εξηγήσει παραπάνω, ο τρόπος το μοντέλο μας να γίνεται αμφίδρομο είναι να θέσουμε True την κατάλληλη παράμετρο και οι κατάλληλες προσαρμογές γίνονται αυτόματα από το μοντέλο μας.

Ουστιαστικά, το bidirectional LSTM έχει τα διπλάσια κύτταρα LSTM από το απλό. Τα μισά διαδίδουν πληροφορία του hidden state προς τα εμπρός, όπως και πριν, ενώ τα άλλα μισά κάνουν το ίδιο αλλά στην αντίστροφη κατεύθυνση.

Η ανάγνωση της ακολουθίας εισόδου τόσο από την αρχή προς το τέλος της όσο και αντίστροφα, μας επιτρέπει να καταλάβουμε το context και τα συμφραζόμενα καλύτερα. Προφανώς αυτό ισχύει και έχει σημασία και στην προφορά των ψηφίων αφού για παράδειγμα τόσο το two όσο και το three ξεκινούν με το ίδιο σύμφωνο ήχο ενώ τελειώνουν με διαφορετικό. Επίσης, με τα αμφίφρομα LSTM μπορούμε να αντιμετωπίσουμε καλύτερα την "απώλεια μνήμης" για μεγάλες ακολουθίες. Προφανώς όταν μία ακολουθία είναι μεγάλη, το context και η πληροφορία της αρχής ξεθωριάζει καθώς ξεδιπλώνουμε το LSTM. Τώρα, όμως, διαβάζοντάς τη και από το τέλος έχουμε καλύτερη συγκράτηση και συνδυασμό πληροφορίας από όλο το μήκος της ακολουθίας. Η αύξηση αυτή της χωριτηκότητας του μοντέλου που περιγράφουμε οδηγεί συνήθως και σε αύξηση της απόδοσής του.

8

Στο τελευταίο Bonus υποερώτημα καλούμαστε να πειραματιστούμε με την τεχνική του packing. Ο τρόπος με τον οποίο λειτουργεί εξηγήθηκε στο υποερώτημα 2. Συνοψίζουμε τα πλεονεκτήματά του:

- Αποφυγή θορύβου από το padding στο dataset
- Αύξηση της ταχύτητας εκπαίδευσης αφού βλέπουμε πολλές μικρές ακολουθίες ταυτόχρονα αντί ξεχωριστά
- Βελτίωση της αριθμητικής ευρωστείας του μοντέλου, αφού όλο και λιγότερα δείγματα (συνεχόμενα κιόλας) είναι μηδενικά και έτσι τα gradients είναι "καθαρότερα"

Αποτελέσματα

Μετά την περιήγηση στα υποερωτήματα μας θα παρουσιάσουμε τα αποτελέσματα των πειραματισμών μας.

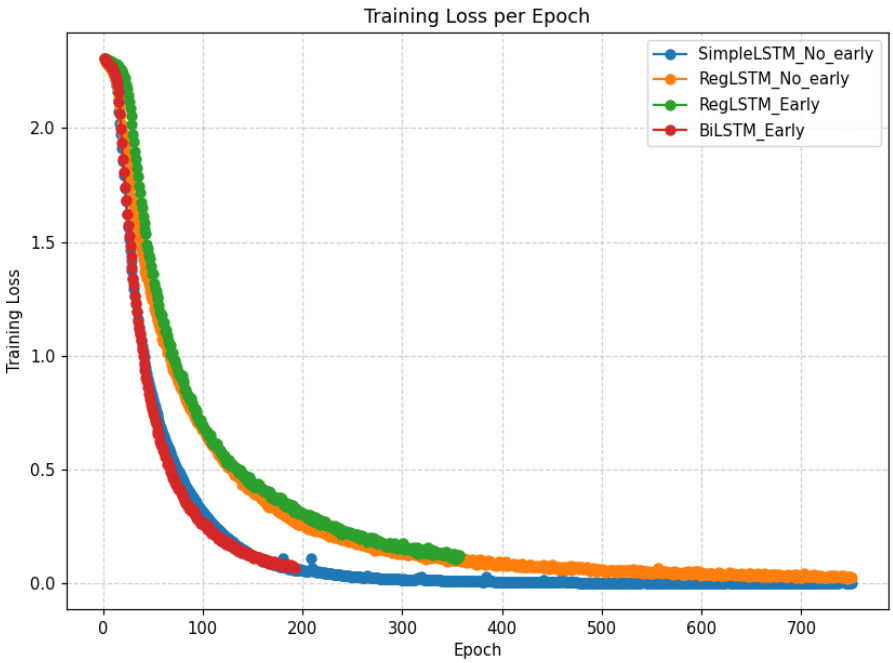


Figure 3: Models' Train Loss

Το να παραθέσουμε τα prints του trainning loss σε κάθε εποχή μπορεί να βοήθησε στο tunning των epochs και του learning rate αλλά δεν έχει νόημα να τα παρουσιάσουμε με λεπτομέρεια στην αναφορά. Αντί αυτού έχουμε κάνει το παραπάνω plot που απεικονίζει τη σύγκλιση όλων των μοντέλων που εκπαιδεύσαμε.

Όπως παρατηρούμε στο διάγραμμα, το early stopping πράγματι λειτουργεί διακόπτοντας την εκπαίδευση των μοντέλων που το χρησιμοποιούν όταν αυτά έχουν σχεδόν σταθεροποιηθεί. Αυτό γλιτώνει πάνω από το μισό χρόνο εκπαίδευσης δίχως να έχει μεγάλη επίπτωση στην απόδοση - όπως θα δούμε σε λίγο. Επίσης, όπως είναι λογικό, τα μοντέλα που χρησιμοποιούν regularization γενικά συγκλίνουν πιο αργά. Αυτό συμβαίνει αφενός λόγω του dropout το οποίο απενεργοποιεί διάφορους νευρώνες καθιστώντας πιο αργή την individual σύγκλισή τους και αφετέρου επειδή το L2 regularization δεν ευνοεί επίσης την ταχύτητα σύγκλισης. Το Bidirectional LSTM χρησιμοποιεί και αυτό regularization αλλά συγκλίνει πολύ γρήγορα λόγω της ευκρίνειας στα gradients που δίνεται από την πιο πλούσια πληροφορία.

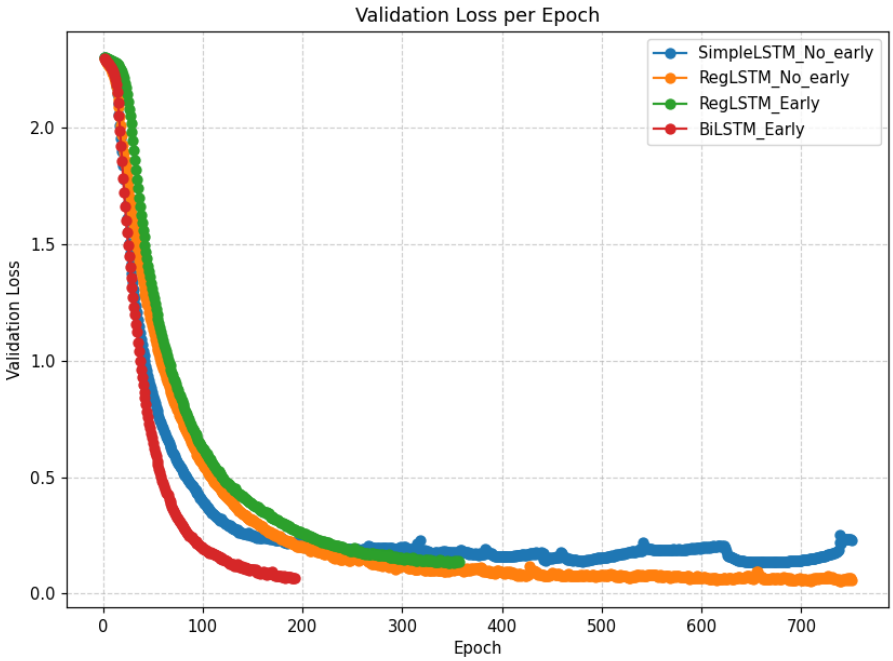


Figure 4: Models' Validation Loss

Γενικά, ένα μοντέλο στο οποίο δεν υπάρχει overfitting, πρέπει να έχει παρόμοια εξέλιξη στην σύγκλιση του validation και του trainning accuracy και πιο σημαντικά, στην κατάσταση σύγκλισης να μην υπάρχει χάσμα μεταξύ στις δύο μετρικές (προφανώς δεν μπορεί ποτέ να έχουμε καλύτερο accuracy στο validation set από ότι στο train αλλά πρέπει να είναι κοντά). Όπως βλέπουμε στο διάγραμμα, στο μόνο μοντέλο στο οποίο αυτό δεν συμβαίνει, δηλαδή βλέπουμε δείγματα overfitting είναι στο απλό μοντέλο δίχως χρήση regularization. Αυτό είναι απολύτως λογικό, αφού η χωρητικότητα του μοντέλου υπερκαλύπτει τις ανάγκες του task και δεν υπάρχει κανένας μηχανισμός αποφυγής της υπερπροσαρμογής του στα δεδομένα εκπαίδευσης. Αυτό δεν συμβαίνει με τα υπόλοιπα μοντέλα που χρησιμοποιείται regularization πράγμα που επιβεβαιώνει την αποτελεσματικότητά του και τη σωστή χρήση του στον κώδικά μας.

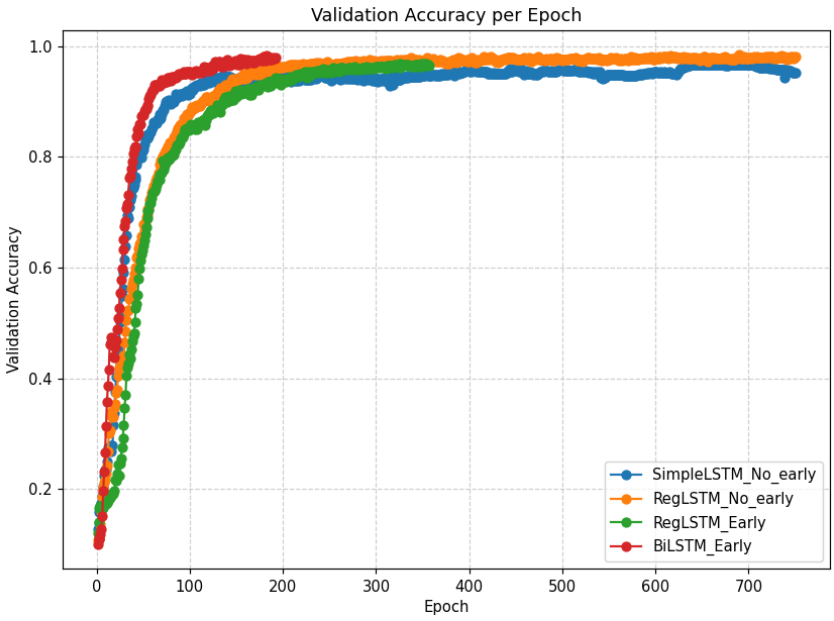


Figure 5: Models' Validation Accuracy

Όπως είναι λογικό, το accuracy στο validation set ακολουθεί αντίστροφη πορεία σύγκλισης από το loss σε όλα τα μοντέλα.

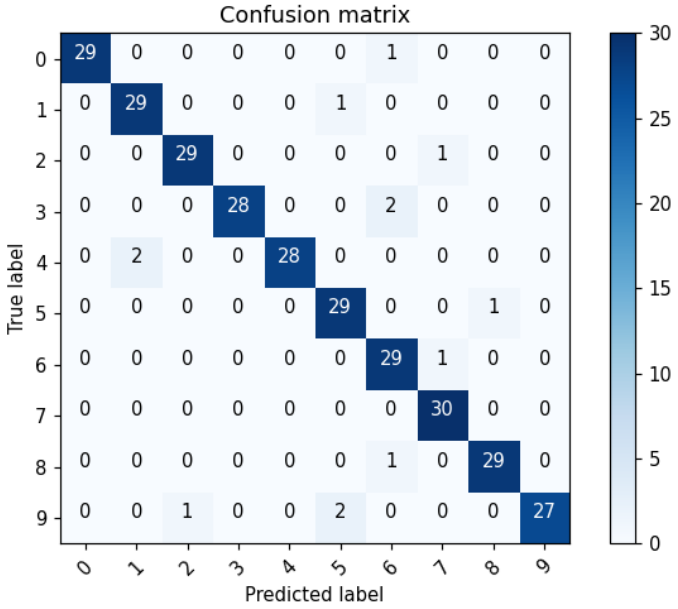


Figure 6: Confusion Matrix

Το confusion matrix στο test set δείχνει πως το καλύτερο μοντέλο μας τα πάει εξαιρετικά αφού κάνει ελάχιστα λάθη σε όλα τα ψηφία.

```
Model training time without packing: 82.526691198349.
Model training time with packing: 288.780015707016.
The best model was SimpleLSTM_No_early
```

Figure 7: Confusion Matrix

References

[1] Jose De la Rosa, Aldonso Becerra, and Efrén González. Speech recognition in a dialog system: from conventional to deep processing a case study applied to spanish. *Multimedia Tools and Applications*, 77:15875–15911, 06 2018.